LAB 16

Automation Stage: a Scheduled, Risk-Checked Forward Test and Live Paper Trading

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	Automation
TSC reference	A7, A8, K7
Duration	25 min
Folder	labs_fullcode/Lab16-paper-trading-bot/
Files	Lab16-paper-trading-bot/run_stage.py, pipeline/runner.py, pipeline/risk.py, Lab16-paper-trading-bot/solution/runner.py, Lab16-paper-trading-bot/solution/risk.py
Prompts	Lab16_Prompts.pdf in this folder (step -> prompt, ready to copy)
Mode	Runs ONLINE with your Alpaca paper keys (.env), or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

The desk is ready to automate the configured EMA 10/30 strategy on SPY. Before any capital is considered, the pipeline must run unattended as a forward test: one decision cycle at a fixed time, completed bars only, pre-trade risk checks, a kill switch, idempotent paper orders and an audit row for every decision - first as a 2025 replay, then live on the Alpaca paper account during class.

Objective

Build pipeline/risk.py and pipeline/runner.py: `python -m pipeline run --once | --loop` with the market clock, risk checks, kill switch and trade log; run a 2025 forward test; run the strategy live on the paper account for a few cycles; and schedule it with cron, launchd or Windows Task Scheduler.

Before You Start

- Open a terminal in the labs_fullcode folder, activate your virtual environment and open your AI coding assistant there (set-up below). The complete, working code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 16	python3 Lab16-paper-trading-bot/run_stage.py (same as: python -m pipeline run)
The complete code	pipeline/risk.py, pipeline/runner.py (reference copy: Lab16-paper-trading-bot/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab16-paper-trading-bot/Lab16_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Build the risk layer and the kill switch (A7)

Ask your assistant for pipeline/risk.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/risk.py with class RiskManager(cfg). kill_switch_on() = the file named by risk.kill_switch_file exists under labs. check(broker, status, symbol, wants_buy, allow_queue=False) returns (ok_to_trade, ok_to_buy, notes): kill switch -> (False, False, ['KILL_SWITCH present - trading halted']); session 'closed' -> blocked with 'market closed' unless allow_queue (then note 'order will queue (GTC) for the next session'); equity / last_equity - 1 <= -risk.max_daily_loss_pct -> no new buys; symbol not in risk.allowed_symbols -> no buy; account trading_blocked -> no buy; otherwise ['all checks passed']. cap_weight(weight, equity, price) applies max_position_pct and max_order_value. kill_main(argv) for `python -m pipeline kill --on | --off | --status`: --on writes the file with a timestamp; --off MOVES it to _state/kill_switch_history/KILL_SWITCH.<timestamp> (an audit record - never delete it). Risk is deterministic code; no model may override it. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline kill --status prints 'KILL SWITCH OFF'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build one decision cycle (A7)

Ask your assistant for decide() and run_cycle() in pipeline/runner.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/runner.py. decide(bars, cfg) = the configured strategy's target from pipeline.signals.target_position on COMPLETED bars only, returning bar_date, close, signal and (for ema_cross) ema_fast and ema_slow. run_cycle(broker, om, risk, bars, run_time, cfg, weight, allow_queue=False, poll_seconds=0): read position and pending shares from the pipeline.execution.OrderManager; wants_buy = signal 1 and nothing held or pending; wants_sell = signal 0 and shares held; run RiskManager.check; if allowed call om.rebalance({symbol: weight}) or om.rebalance({symbol: 0.0}) with tag = the bar date (one decision per bar, so a double-fired job cannot double-trade); otherwise action BLOCKED if a trade was wanted; return a log row with run_time, bar_date, symbol, close, ema_fast, ema_slow, signal, position_before, action, qty, order_id, status, fill_price, equity, checks. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: run_cycle returns a dict with the 15 log fields.

Step 3: Build the forward-test replay and the logs (A8)

Ask your assistant for replay(), once() and the log writers.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/runner.py add replay(cfg, start, end): offline forward test on PaperBrokerSim (state_file bot_sim.json, reset) - for each 2025 session set the simulator clock to 09:35, mark the price at today's open, run_cycle on bars up to YESTERDAY's close with weight risk.max_position_pct, then process the day's bar and roll end_of_day; stop if the kill switch appears. Write _outputs/trade_log.csv (fresh, one row per session) and _outputs/Lab16_equity_log.csv (date, equity, position, close), print every non-HOLD row, the number of orders, start and end equity and SPY buy and hold over the same days, and save chart Lab16_forward_test.png. once(cfg): one cycle; offline it acts at the open after the last bundled bar and appends to trade_log.csv; online it appends to _outputs/live_paper_log.csv. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline run --help lists --once, --loop, --live-paper, --interval, --max-cycles, --start and --end.

Step 4: Add the live paper loop and scheduling (A7)

Ask your assistant for live_loop(), main() and schedule_main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/runner.py add live_loop(cfg, interval, max_cycles): on the Alpaca PAPER account (or the simulator with a clear message if no keys) run up to max_cycles cycles: print the clock line, stop with 'KILL SWITCH ON - no orders sent; loop stopped' if the kill switch exists, run_cycle with weight live_paper.max_position_pct, allow_queue=True and poll_seconds live_paper.poll_seconds online (the OrderManager sends extended-hours limit orders outside regular hours), append to _outputs/live_paper_log.csv, print the row and every position with unrealised P&L, then sleep interval seconds; finish with the dashboard address. main(argv): --loop --live-paper -> live_loop; --loop offline -> replay; --loop online -> live_loop; otherwise once. schedule_main(argv) for `python -m pipeline schedule`: print and write to _outputs/Lab16_schedule/ a crontab line with CRON_TZ=America/New_York for schedule.run_at_exchange_time Mon-Fri (with the Singapore time in US summer and winter), a launchd plist (com.lioncitycapital.pipeline, WorkingDirectory labs) and a Windows schtasks command, each running `python -m pipeline run --once` from labs and appending to _outputs/scheduler.log. Do not install anything. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline schedule --help works.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Run the 2025 forward test (A8, K7)

Run: python -m pipeline run --loop (same as: python3 Lab16-paper-trading-bot/run_stage.py --loop). Offline this replays 2025 day by day; each order fills at that day's open in the simulator.

Check: 250 sessions, 5 orders: BUY 159 after the 2025-01-21 close, SELL after 2025-02-27, BUY 166 after 2025-05-02, SELL after 2025-11-19, BUY 163 after 2025-11-28; equity USD 100,000.00 -> 116,249.79 (+16.25%) versus SPY buy and hold +17.06%; trade_log.csv with 250 rows; Lab16_forward_test.png.

Step 6: Audit the decision log (A8)

Open _outputs/trade_log.csv. Check bar_date, signal, action, qty, status, fill_price, equity and checks for one BUY day and one HOLD day.

AI prompt - paste into your AI coding assistant
Read labs_fullcode/_outputs/trade_log.csv and write an audit summary for the compliance officer: number of sessions, decisions by action, every order with its bar date, fill price and equity after the fill, and any row where a risk check did not say 'all checks passed'. Do not change any file.

Check: 250 rows: 5 BUY/SELL rows with status filled and 245 HOLD rows; 249 rows say 'all checks passed' and one (2025-01-27) says 'daily loss -2.05% breaches 2% limit - no new buys' - a HOLD day with the position already held, so the rule blocked nothing.

Step 7: Test the kill switch (A7)

Run: python -m pipeline kill --on, then python -m pipeline run --once, then python -m pipeline kill --off.

Check: The --once row shows checks 'KILL_SWITCH present - trading halted' (no order can be sent); --off prints 'moved to _state/kill_switch_history/KILL_SWITCH.<timestamp>' - the halt is kept as an audit record.

Step 8: Run the strategy LIVE on the paper account (A7, A8, K7)

With paper keys in .env, during class run: python -m pipeline run --loop --interval 60 --live-paper --max-cycles 5. Watch decision -> order -> fill -> position -> log for each cycle. After two or three cycles, in a SECOND terminal run python -m pipeline kill --on and watch the next cycle halt. Then run python -m pipeline kill --off and open _outputs/live_paper_log.csv and the Alpaca dashboard.

Check: Cycle 1 sends one SPY buy for 10% of equity (an extended-hours LIMIT DAY order in Singapore class hours; about 14 shares on USD 100,000) and prints its lifecycle and the position; later cycles HOLD ('buy order ... still working' or position held); after kill --on: 'KILL SWITCH ON - no orders sent; loop stopped'. Without keys the same loop runs on the simulator (at a weekend the buy queues as a GTC limit order).

Step 9: Schedule the runner (A7)

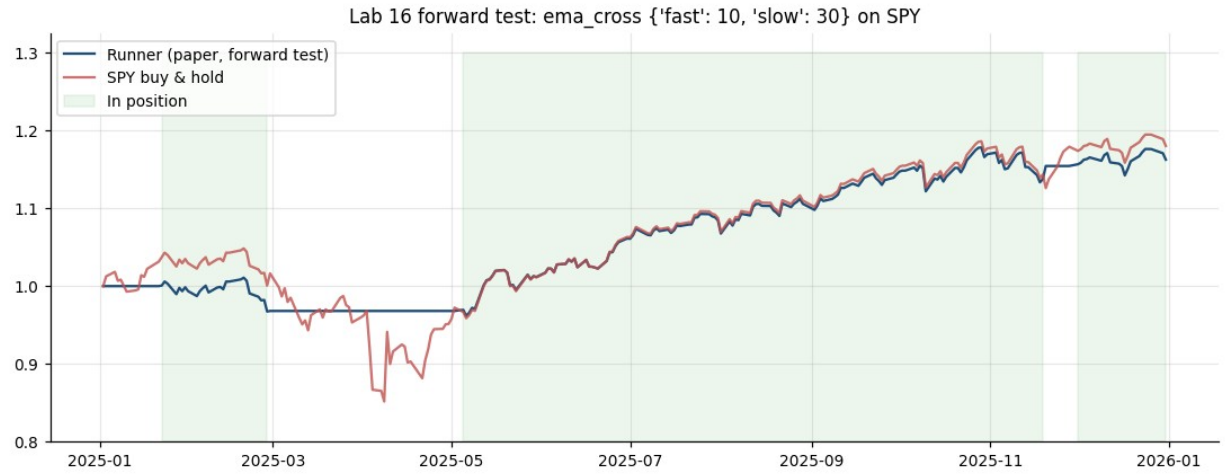
Run: python -m pipeline schedule and read the three entries. Only install one if your instructor approves (crontab -e, launchctl load, or schtasks in a Command Prompt).

Check: One cycle at 09:35 New York = 21:35 Singapore (US summer) / 22:35 (US winter), Mon-Fri; files crontab.txt, com.lioncitycapital.pipeline.plist and schtasks.cmd in _outputs/Lab16_schedule/; nothing is installed.

Expected Output (Full Code)

python -m pipeline run --loop (offline 2025 replay): 250 sessions, 5 orders (BUY 159, SELL, BUY 166, SELL, BUY 163), equity USD 100,000.00 -> 116,249.79 (+16.25%) versus SPY +17.06%; trade_log.csv with 250 rows; kill switch halts the next cycle and is archived on --off; run --loop --live-paper sends a 10% SPY order as an extended-hours limit during Singapore hours and stops on the kill switch; schedule entries for 09:35 New York (21:35 / 22:35 Singapore); chart _outputs/Lab16_forward_test.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.



Lab 16 output (labs_fullcode/_outputs/Lab16_forward_test.png), offline mode on the bundled data.

Guide: Paper Trading on Alpaca, Step by Step - Manual and Automated

First place one order by hand to see the order lifecycle, then let the pipeline place orders automatically (Labs 8 and 16).

1. Place a manual paper order

In the Alpaca dashboard select the Paper Trading account (upper left), search the symbol (for example SPY) and open its page. Set the side, quantity, order type and time in force, then press Buy.

2. Follow the order lifecycle

Open the Orders view and watch the status move from new / accepted to filled (or cancelled). Open Positions to see the holding, cost basis and unrealised P&L.

3. Know the trading hours from Singapore

The US regular session runs 21:30-04:00 Singapore time (22:30-05:00 in the northern winter). Outside it, Alpaca's 24/5 trading accepts only LIMIT orders with extended hours enabled (time in force DAY or GTC), at lower liquidity; a market order placed while the market is closed waits for the next session.

4. Let the pipeline place the orders

In the labs folder run: python -m pipeline execute (Lab 8). The order manager reads the Alpaca clock, compares the target position with the actual one, submits the paper order, polls its status and prints the fill.

5. Run the strategy automatically

Run: python -m pipeline run --loop --interval 60 --live-paper (Lab 16). Each cycle fetches bars, computes the signal, runs the risk checks, places or skips the paper order and writes a row to the trade log.

6. Check the results in the dashboard

Refresh the Alpaca dashboard: the bot's orders appear under Orders and its holdings under Positions, matching labs/_outputs/trade_log.csv.

7. Stop it safely

Run: python -m pipeline kill --on. The next cycle halts before any order is sent. Cancel any working orders and, if you wish, close positions from the dashboard.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

From Singapore the US regular session is 21:30-04:00 (22:30-05:00 in northern winter); class-time orders use Alpaca's 24/5 extended-hours limit orders.

Debrief

The runner is a software agent coded from a selected model (A7): signal on completed bars, deterministic risk limits, idempotent execution, a kill switch and an audit log. The forward test and the live paper loop check the actions it takes and their outcomes on unseen data (A8), including the real effect of whole shares, fills and session rules (K7).

Troubleshooting

Symptom	What to do
Action BLOCKED with 'market closed'	Expected for --once at a weekend online; the live loop queues a GTC order instead (allow_queue).
No trades in the replay	Check that labs_fullcode/KILL_SWITCH does not exist (python -m pipeline kill --status).
A cron job ran twice	The client_order_id is unique per decision bar, so the second run finds the order and sends nothing.
macOS cron fires at the wrong hour	macOS cron ignores CRON_TZ; use the Singapore-time line or launchd.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.